

Análise Comparativa de Algoritmos de Pathfinding para NPCs em Jogos 2D em Ambiente Web: A*, Dijkstra, BFS, DFS e Jump Point Search

Jeandeson Souza Nascimento¹

¹Curso de ciência da computação – Centro Universitário Jorge Amado (UNIJORGE)
Caixa Postal 15.064 – 41720-000 – Salvador – SA – Brazil

jeandeson.nascimento@bol.com.br

Abstract. *This work presents the development and comparative analysis of pathfinding algorithms for non-playable characters (NPCs) in 2D games, implemented in JavaScript/TypeScript using the HTML Canvas API. Five algorithms were implemented from scratch: A*, Dijkstra, BFS, DFS, and Jump Point Search (JPS), an optimization of A* that reduces the number of nodes evaluated through directional pruning and jump points. A functional interactive simulator was developed and made available online, enabling real-time visualization of pathfinding in a top-down game. Benchmark tests were conducted across multiple grid configurations, obstacle densities, and NPC counts, with each scenario executed 10 times to stabilize timing measurements. The results showed that A* achieved mean execution times on the order of hundredths of a millisecond per search in 20×20 grids with 20% obstacle density — comfortably within the 16ms budget of a single frame at 60fps — and that JPS reduced the number of expanded nodes by more than 99% in high-density grids. With 50 simultaneous NPCs, all five algorithms completed their searches in under 2ms (well within the 16ms frame budget); notably, JPS's large reduction in expanded nodes did not translate into the lowest wall-clock time on small, sparse grids, where the per-jump scanning overhead dominates — its time advantage appears only in dense grids. The work contributes to the analysis of pathfinding performance in web environments, an under-explored context in the literature.*

Resumo. *Este trabalho apresenta o desenvolvimento e a análise comparativa de algoritmos de busca de caminhos (pathfinding) para personagens não jogáveis (NPCs) em jogos 2D, implementados em JavaScript/TypeScript com a API Canvas do HTML. Foram implementados do zero cinco algoritmos: A*, Dijkstra, BFS, DFS e Jump Point Search (JPS), uma otimização do A* que reduz o número de nós avaliados por meio de poda direcional e pontos de salto. Um simulador interativo e funcional foi desenvolvido e disponibilizado online, permitindo a visualização em tempo real do pathfinding em um jogo top-down. Testes de benchmark foram realizados em múltiplas configurações de grade, densidades de obstáculos e quantidade de NPCs, com cada cenário executado 10 vezes para estabilizar as medições de tempo. Os resultados demonstraram que o A* obteve tempos médios de execução na ordem de centésimos de milissegundo por busca em grades 20×20 com densidade 0,2 — folgadoamente dentro do orçamento de 16ms de um quadro a 60fps — e que o JPS reduziu em mais de*

99% o número de nós expandidos em grades de alta densidade. Com 50 NPCs simultâneos, todos os cinco algoritmos concluíram as buscas em menos de 2ms (bem dentro do orçamento de 16ms); notavelmente, a grande redução de nós do JPS não se traduziu no menor tempo de relógio em grades pequenas e esparsas, onde o custo das sondagens por salto predomina — sua vantagem em tempo só aparece em grades densas. O trabalho contribui para a análise de desempenho de algoritmos de pathfinding em ambientes web, um contexto pouco explorado na literatura.

1. Introdução

Jogos eletrônicos são sistemas que combinam regras formais com mundos fictícios, criando resultados quantificáveis que dependem do esforço do jogador Juul (2005). Nessa perspectiva, Salen and Zimmerman (2004) descreve os jogos como sistemas compostos por regras, objetivos e resultados, cujos elementos de mecânica, estética e *feedback* estruturam a experiência de jogo.

Com a popularização de motores gráficos modernos como Unity e frameworks web como Phaser.js, desenvolvedores independentes (*indie*) têm acesso facilitado a recursos de Inteligência Artificial (IA) já integrados. Entretanto, a compreensão de qual algoritmo de busca de caminhos utilizar em diferentes cenários — e qual seu custo real de desempenho em ambientes web — permanece uma questão relevante e pouco documentada na literatura.

Este trabalho analisa comparativamente cinco algoritmos de *pathfinding* — A*, Dijkstra, BFS, DFS e Jump Point Search (JPS) — implementados em JavaScript/TypeScript sem dependências externas, avaliando seu desempenho em um ambiente de execução web real. O JPS, proposto por Harabor and Grastien (2011), é uma otimização do A* que reduz o espaço de busca por meio de poda direcional, mantendo a garantia de caminho ótimo.

1.1. Objetivo Geral

Analisar comparativamente o desempenho dos algoritmos de busca de caminhos A*, Dijkstra, BFS, DFS e Jump Point Search (JPS) em um ambiente 2D com obstáculos, por meio do desenvolvimento de um simulador interativo funcional em JavaScript/TypeScript, avaliando métricas de eficiência computacional em ambiente web.

1.2. Objetivos Específicos

Os objetivos específicos propostos nessa pesquisa são:

1. Estudar os princípios dos algoritmos A*, Dijkstra, BFS, DFS e Jump Point Search e suas aplicações em ambientes 2D com grades uniformes.
2. Implementar os cinco algoritmos do zero, sem bibliotecas externas, sobre uma estrutura de grade com geração procedural de obstáculos.
3. Desenvolver e disponibilizar um simulador interativo funcional que permita a visualização em tempo real do *pathfinding* em um jogo *top-down*, acessível por URL pública.

4. Avaliar e comparar a eficiência dos algoritmos por meio de métricas quantitativas — tempo de execução, nós visitados, comprimento do caminho e taxa de sucesso — em cenários variados de densidade de obstáculos, quantidade de NPCs simultâneos, congestionamento e objetivos dinâmicos.
5. Validar a admissibilidade da heurística de Manhattan utilizada no A* e no JPS, demonstrando a garantia de otimalidade dos caminhos encontrados.

1.3. Justificativa

Em jogos digitais, principalmente em jogos de exploração, o movimento inteligente é essencial para proporcionar uma experiência satisfatória ao jogador Moon et al. (2022). O A* é mundialmente reconhecido por ser um algoritmo preciso e eficiente para busca de caminhos em cenários complexos, amplamente utilizado em robótica, navegação e jogos digitais Hart et al. (1968).

Embora o A* seja consolidado, estudos como Harabor and Grastien (2011) demonstraram que, em grades uniformes, é possível reduzir drasticamente o número de nós avaliados por meio do Jump Point Search (JPS), uma técnica de poda que elimina simetrias de caminho sem comprometer a otimalidade. A comparação empírica entre o A* padrão e o JPS em ambiente web — onde o motor JavaScript (V8) introduz variações de desempenho não observadas em ambientes nativos compilados — representa uma lacuna identificada na revisão da literatura realizada neste trabalho, visto que a maioria dos estudos de *pathfinding* reporta experimentos conduzidos em C++ ou ambientes de compilação nativa.

A justificativa científica deste trabalho reside, portanto, na análise de desempenho comparativa em um ambiente de execução específico (JavaScript/navegador web) com restrições reais de hardware, distinta da maioria dos estudos que utilizam C++ ou ambientes nativos. Tal análise é relevante para desenvolvedores independentes (*indie*) que implementam jogos 2D diretamente no navegador e carecem de referências de desempenho nesse contexto.

O projeto também se justifica pela entrega de um artefato funcional acessível — um simulador interativo hospedado online — que permite à banca e à comunidade verificar e reproduzir os resultados diretamente pelo navegador, sem necessidade de instalação.

2. Referencial Teórico

2.1. Algoritmos de Pathfinding

Os algoritmos de pathfinding permitem execução de movimentação eficaz para os agentes em ambientes virtuais. São esses algoritmos que possuem a responsabilidade de encontrar os caminhos considerados mais curtos e eficientes entre um ponto e outro, podendo levar em consideração mudanças no cenários, como obstruções e restrições. Os algoritmos mais conhecidos são: Dijkstra Dijkstra (1959), o Breadth-First Search (BFS), o Depth-First Search (DFS) Cormen et al. (2009) e o algoritmo A* (A-Star) Hart et al. (1968), que foi considerado um dos mais eficazes Millington and Funge (2009). Revisões sistemáticas recentes confirmam que esses algoritmos permanecem os mais utilizados em jogos digitais Al-Salami et al. (2022), com estudos comparativos demonstrando que a escolha do algoritmo impacta diretamente o desempenho em cenários específicos — incluindo labirintos e grades com obstáculos Salem et al. (2024).

2.2. Algoritmo de Dijkstra

O algoritmo de Dijkstra, concebido por Edsger W. Dijkstra em 1956 e publicado em 1959 Dijkstra (1959), é um dos algoritmos mais importantes para encontrar o menor caminho entre nós de um grafo. Ele é a base e é extremamente fundamental para entender outros algoritmos considerados mais avançados como o A*. Esse algoritmo soluciona o problema de caminho mais curto, conhecido também por problema do caminho mínimo que consiste em encontrar o menor caminho entre grafos de pesos não negativos. O funcionamento ocorre expandindo de forma progressiva o grupo de nós onde as distâncias mínimas do nó origem são conhecidas, continuamente buscando escolher o nó com a distância mínima conhecida que seja menor para expandir primeiro Millington and Funge (2009).

2.3. Breadth-First Search (BFS)

O Breadth-First Search é um algoritmo de busca que itera sobre todos os nós vizinhos no nível atual antes de prosseguir para o próximo. Isso permite encontrar o menor caminho em termos de número de passos. A busca se inicia no nó raiz, explorando o primeiro nível da esquerda para a direita, continua para o nível dois e segue esse fluxo. A vantagem na utilização desse algoritmo é que, em grafos de custo uniforme, ele garante encontrar um caminho de menor número de passos: como explora os nós em camadas crescentes de distância a partir da origem, o objetivo é alcançado pela primeira vez já por um caminho mais curto. Note-se que o BFS retorna *um* caminho ótimo, e não todas as soluções possíveis. Entretanto, a desvantagem que ele apresenta é a grande utilização de memória, pois armazena os nós na árvore, necessitando de um uso extensivo de memória Cormen et al. (2009).

De acordo com Rahim et al. (2018), o BFS é eficaz em planos cartesianos, ao determinar rotas ótimas e curtas, em que cada ponto é calculado como possível caminho. Dessa forma, ele utiliza grandes buscas, adentrando em cada nó de todos níveis, isso gera diversas rotas possíveis, desde a menor localizando (o caminho mais curto) até a maior (caminho mais longo), ele se apresenta flexível em diversos cenários de roteamento. Experimentos mostram que o BFS se sai muito bem quando utilizado nesse contexto. O BFS é um destaque quando utilizado em grids cartesianos, e é muito eficiente para problemas de pathfinding em ambientes virtuais.

2.4. Depth-First Search (DFS)

O Depth-First Search (DFS), ou busca em profundidade, é um algoritmo que adentra o mais fundo possível ao decorrer de cada ramo antes retornar (backtracking). A diferença entre ele e o BFS, que dá prioridade à níveis vizinhos, o DFS continua até o fim, para só depois seguir outras alternativas, fazendo algo parecido com uma busca em árvore ou grafo, só que de maneira profunda. O algoritmo inicia seu processo em um nó raiz. visita de forma recursiva todos os nós vizinhos ainda não visitados, em seguida marcando o nó para não causar redundância, e por fim retornando quando não há mais caminhos possíveis. A principal vantagem do DFS é sua implementação simples, ele utiliza pilha para guardar apenas o caminho vigente, diferente do BFS que armazena todos nós referente a um nível Cormen et al. (2009). Porém, sua desvantagem é a não garantia de encontrar o caminho mais curto em grafos não ponderados, assim resultando em soluções subótimas, e em casos onde os nós são muito profundos, o uso demasiado de recursão levar a estouro

de pilha que é quando uma função recursiva que em teoria possui fim, mas por limitações, causa um estouro do buffer da pilha de chamadas na prática.

De acordo com Tarjan (1972), o DFS tem seu valor ilustrado como estratégia de *backtracking*, utilizando de exemplos que mostram suas aplicações em algoritmos para grafos. Tarjan mostra um DFS aprimorado utilizado para obter componentes que estão profundamente ligados (SCCs) em grafos dirigidos e uma outra versão para elementos que estão biconectados em grafos não dirigidos. Essas versões usam o DFS para estabelecer uma árvore (spanning tree), concedendo valores e tempos de resolução de *low-link* aos nós, isso permite a descoberta de ciclos e pontos de articulação em tempo linear $O(V + E)$, sendo V a quantidade de vértices e E a de arestas. Para constantes k_i essa complexidade pode ser descrita por $k_1V + k_2E + k_3$, fazendo com que DFS seja um instrumento poderoso na solução de problemas de conectividade em grafos, com aplicabilidade em redes, ordenação topológica e em teste de planaridade. Tarjan trouxe nessa abordagem uma base eficiente para algoritmos que usam grafos, tendo influência em desenvolvimentos subsequentes dentro da área de inteligência artificial e também computação gráfica.

No contexto de pathfinding para agentes em jogos 2D, o DFS puro apresenta uma limitação prática fundamental: sem orientação heurística, ele expande vizinhos na ordem fixa retornada pela grade (norte, sul, oeste, leste) e pode percorrer toda a área alcançável antes de encontrar o objetivo. Em testes realizados durante o desenvolvimento deste trabalho, o DFS puro gerou caminhos altamente sinuosos que tornavam o comportamento dos agentes inviável em tempo real — o NPC se deslocava em direção oposta ao objetivo por longos trechos antes de redirecionar. Por essa razão, a implementação adotada neste trabalho utiliza uma variante denominada ***Greedy DFS***: antes de empilhar os vizinhos, eles são ordenados de forma decrescente pela distância Manhattan ao objetivo. Como a pilha é LIFO, o vizinho mais próximo do objetivo fica no topo e é expandido primeiro, direcionando a busca sem alterar a estrutura de pilha nem introduzir garantia de otimalidade — características fundamentais do DFS são preservadas. Essa variante aproxima-se da ideia da busca gulosa (*greedy best-first search*) descrita por Russell and Norvig (2010), porém preservando a estrutura de pilha do DFS: a ordenação heurística apenas direciona a escolha local dos sucessores, sem manter uma fronteira de prioridade global como faria a busca gulosa pura. Trata-se, portanto, de uma busca em profundidade guiada por heurística, que não oferece garantia de otimalidade. A implicação direta é que o ***Greedy DFS*** visita menos nós e executa mais rapidamente que A* e BFS, porém encontra caminhos sistematicamente maiores que o ótimo, evidenciando o conflito entre velocidade de busca e qualidade de caminho.

2.5. O Algoritmo A*

O A* (A-star), difundido por Hart, Nilsson e Raphael em 1968, representa um marco essencial na área de busca heurística e *pathfinding* Hart et al. (1968). Ele concilia o algoritmo Dijkstra com uma busca informada de forma eficaz. O A* rapidamente se tornou o algoritmo mais usado em jogos digitais, sobretudo devido ao seu equilíbrio entre eficiência e precisão, sendo especialmente eficaz em ambientes estruturados como grids.

O algoritmo A* é uma extensão do algoritmo de Dijkstra, que incorpora uma heurística para estimar o custo restante até o objetivo, tornando a busca mais eficiente. Ele utiliza duas funções principais: $g(n)$, que representa o custo do caminho do ponto

inicial até o nó n , e $h(n)$, que é a estimativa heurística do custo de n até o destino. A função $f(n) = g(n) + h(n)$ é usada para priorizar a busca pelos caminhos mais promissores.

2.6. Funcionamento do algoritmo A*

O A* é um algoritmo de busca em grafos que localiza o menor caminho com menor custo entre um nó de partida e um nó final, também chamado de nó objetivo. Sua maior novidade é o uso de uma função que estima o restante do custo até o nó final, isso permite que o algoritmo dê prioridade na busca de caminhos melhores e mais promissores Hart et al. (1968).

A função heurística de avaliação do A* é simbolizada como $f(n) = g(n) + h(n)$, onde:

- $f(n)$: é o valor de avaliação total do nó n .
- $g(n)$: é o custo real do caminho do nó inicial até o nó n .
- $h(n)$: é a função heurística que estima o custo do caminho mais curto de n até o objetivo.

A estrutura do processo do A* é definida por:

1. Inicialização:
2. Iteração principal:
3. Seleciona-se o nó n do conjunto aberto com o menor valor de $f(n)$
4. Se n for o nó objetivo, reconstrói-se e retorna-se o caminho
5. Caso contrário, remove-se n do conjunto aberto, adiciona-o ao conjunto fechado
6. Para cada vizinho m de n :
 - Se m estiver no conjunto fechado, ignora-o
 - Caso contrário, calcula-se $g(m) = g(n) + \text{custo}(n, m)$
 - Se m não estiver no conjunto aberto, adiciona-o
 - Se m já estiver no conjunto aberto com um valor g maior, atualiza-se seu valor g e seu pai
7. Reconstrução do caminho: Após encontrar o objetivo, o caminho é reconstruído seguindo os ponteiros de pai do nó objetivo até o nó inicial.

A heurística a ser utilizada é de extrema importância para que o A* tenha um bom desempenho. Em ambientes 2D como jogos digitais com grade, as heurísticas mais usadas são:

- Distância de Manhattan ($|x1 - x2| + |y1 - y2|$): Para grades com movimento limitado às quatro direções cardeais.
- Distância Euclidiana ($\sqrt{(x1 - x2)^2 + (y1 - y2)^2}$): Eficiente para quando o movimento acontecer em qualquer direção.
- Distância de Chebyshev ($\max(|x1 - x2|, |y1 - y2|)$): útil quando o movimento diagonal tem o custo igual ao cardeal.

Millington and Funge (2009) evidencia que a heurística deve ser aceitável (nunca superestimar o custo real até o objetivo) e firme (para qualquer nó n e seu sucessor n' , $h(n) \leq \text{custo}(n, n') + h(n')$). Isso irá garantir que o funcionamento do A* resultará no melhor caminho.

Validação da admissibilidade da heurística de Manhattan (Objetivo Específico 5): Em uma grade com movimentação 4-direcional, cada passo cobre exatamente

uma unidade em uma direção cardinal (norte, sul, leste ou oeste). Para deslocar um agente do nó $n = (x_n, y_n)$ ao objetivo $g = (x_g, y_g)$, qualquer caminho válido precisa realizar pelo menos $|\Delta x| + |\Delta y|$ movimentos, pois cada passo reduz a distância horizontal ou vertical em no máximo 1 unidade. Portanto, para todo nó n e toda configuração de obstáculos transitável:

$$h(n) = |x_n - x_g| + |y_n - y_g| \leq \text{custo_real}(n, g) \quad (1)$$

A desigualdade é estrita sempre que obstáculos forçam desvios, e iguala-se ao custo real apenas quando o caminho reto está completamente livre. Como $h(n)$ nunca superestima o custo real, a heurística de Manhattan é *admissível* Hart et al. (1968). A condição de firmeza (consistência) também se verifica: para dois nós adjacentes n e n' separados por um passo cardinal de custo 1, tem-se $h(n) \leq 1 + h(n')$ pela desigualdade triangular da norma L_1 . Essas duas propriedades — admissibilidade e consistência — garantem que o A^* , ao utilizar a distância de Manhattan como heurística, sempre encontra o caminho de custo mínimo Millington and Funge (2009). A variante 4-direcional do JPS aqui implementada, por reusar a mesma heurística admissível, preserva essa otimalidade na prática. Os resultados experimentais deste trabalho corroboram essa garantia: em todos os cenários testados, A^* , Dijkstra, BFS e JPS produziram caminhos de comprimento idêntico (39 células em grades 20×20 com densidade 0,2 e 0,4), confirmando empiricamente a otimalidade.

2.7. Otimizações e Variantes do A^*

Rabin (2015) discute diversas otimizações do A^* para jogos 2D, entre as quais:

- **Weighted A^* (busca com peso)**: multiplica a heurística por um fator $w \geq 1$, acelerando a busca ao custo de uma subotimalidade limitada — o caminho encontrado custa no máximo w vezes o ótimo Pohl (1970).
- **Anytime A^*** : retorna rapidamente uma solução inicial subótima e a refina de forma incremental enquanto houver tempo de processamento, reduzindo progressivamente o limite de subotimalidade Barbehön and Chen (1995).
- **Busca em tempo real ($RTA^*/LRTA^*$)**: limita a quantidade de computação por movimento, intercalando planejamento e ação para garantir tempo de resposta constante, adequado a sistemas em tempo real Korf (1990).
- **Hierarchical Pathfinding (HPA^*)**: divide o mapa em regiões e pré-calcula caminhos entre elas, reduzindo a complexidade em mapas muito grandes Botea et al. (2004).

Entre as otimizações mais relevantes para grades uniformes destaca-se o **Jump Point Search (JPS)**, proposto por Harabor and Grastien (2011). O JPS é uma aceleração do A^* que explora a simetria de caminhos em grades de custo uniforme: ao percorrer uma linha reta, múltiplos caminhos de mesmo custo são simétricos e apenas um precisa ser avaliado. O algoritmo aplica regras de *poda* que eliminam vizinhos redundantes e salta ao longo de linhas retas até encontrar um **ponto de salto** — uma célula que possui *vizinhos forçados*, ou seja, vizinhos cujo caminho ótimo a partir do nó pai obrigatoriamente passa pelo nó atual devido a um obstáculo adjacente.

Formalmente, na movimentação 4-direcional (cardinal), um nó x alcançado na direção $\vec{d}$ possui um vizinho forçado n se:

1. n é transitável (sem obstáculo);
2. o nó imediatamente "atrás" de n em relação a $\vec{d}$ não é transitável (há um obstáculo que impede alcançar n sem passar por x).

O JPS canônico mantém a completude e a otimalidade do A*: quando a heurística é admissível, o caminho encontrado é garantidamente ótimo, e a poda reduz o número de nós inseridos na lista aberta, acelerando a busca em grades densas ou de grande dimensão. Cabe destacar que a formulação original de Harabor and Grastien (2011) pressupõe movimento em **8 direções** (incluindo diagonais), nas quais a quebra de simetria entre caminhos diagonais equivalentes é o cerne do método. No presente trabalho, o JPS foi **adaptado para movimentos 4-direcionais** (cardeais) com heurística de Manhattan, conforme detalhado na Seção 3. Por se tratar de uma adaptação, a otimalidade dessa variante não é herdada diretamente da prova de Harabor e Grastien, sendo **validada empiricamente** em todos os cenários de teste (Seção 4).

2.8. Vantagens e Limitações

Em comparação com os demais algoritmos de *pathfinding*, o A* mantém-se equilibrado quanto à garantia do caminho mais curto, ao uso de memória e à eficiência computacional. O Dijkstra oferece caminho ótimo, porém com eficiência inferior e uso elevado de memória; o BFS também garante o caminho ótimo, mas apenas em grafos com pesos uniformes e com alto uso de memória; e o DFS, por sua vez, pode não entregar o melhor caminho, embora consuma pouca memória. O A* apresenta superioridade em diversos aspectos: encontra o caminho ótimo quando utiliza heurística admissível, é mais eficiente que o Dijkstra por explorar informação sobre o objetivo, é flexível em diferentes representações de mapa e admite adaptações por meio de otimizações.

Entretanto, o A* também apresenta limitações: elevado uso de memória em mapas grandes, dependência de uma heurística de boa qualidade para apresentar bom desempenho, e alto custo computacional quando há muitos agentes em paralelo.

Hart et al. (1968) no seu artigo original observa que o A* possui uma constância ótima entre totalidade do Dijkstra e a eficácia da busca heurística, e isso é o que explica sua grande popularidade em ambientes 2D e jogos digitais, entre outras aplicabilidades do pathfinding.

3. Metodologia

3.1. Ferramentas e Tecnologias Utilizadas

A escolha de JavaScript/TypeScript como linguagem principal justifica-se por fatores técnicos e de acessibilidade. JavaScript é executado em praticamente qualquer plataforma com navegador web, eliminando barreiras de compatibilidade e permitindo que a banca avaliadora acesse o simulador diretamente por URL, sem instalação. TypeScript acrescenta tipagem estática ao JavaScript, reduzindo erros em tempo de compilação e melhorando a manutenibilidade do código em sistemas complexos como algoritmos de pathfinding.

Os motores JavaScript modernos (V8 no Chrome/Node.js e SpiderMonkey no Firefox) aplicam compilação JIT (*Just-In-Time*), tornando a linguagem adequada para algoritmos computacionalmente intensivos como o A* e o JPS. A renderização foi feita

exclusivamente com a API Canvas 2D nativa do HTML, sem frameworks de jogo, para minimizar variáveis externas nos experimentos de desempenho.

O ambiente de desenvolvimento utilizou TypeScript 5 com o empacotador Vite 7. Nenhuma biblioteca de pathfinding externa foi utilizada — todos os algoritmos foram implementados do zero sobre estruturas de dados nativas da linguagem.

3.2. Ambiente de Testes e Reprodutibilidade

Para garantir a reprodutibilidade dos resultados, os benchmarks foram executados sob as seguintes condições:

- **Hardware:** Intel Core i5-10300H, 16 GB RAM, SSD NVMe.
- **Navegador:** Google Chrome (motor V8/Blink), executado em modo *headless* e dirigido de forma automatizada via Chrome DevTools Protocol. O benchmark roda em uma página isolada (`benchmark.html`), sem o laço de renderização do jogo, para que a cronometragem não dispute CPU com o *canvas*.
- **Sistema operacional:** Windows 11 Pro.
- **Método de medição:** `performance.now()` World Wide Web Consortium (2023). Como o Chrome limita a resolução desse relógio a cerca de 100 microssegundos (mitigação de segurança contra Spectre), uma busca isolada — que dura menos que esse piso — seria registrada apenas como ruído de quantização. Para contornar essa limitação, adota-se **medição em lote**: cada unidade de trabalho (uma busca, ou o conjunto de N buscas no caso multi-NPC) é repetida K vezes dentro de uma única janela cronometrada, com K calibrado automaticamente até a janela exceder 25 ms; o tempo reportado por unidade é a janela dividida por K . Dessa forma, mesmo buscas de poucos microssegundos tornam-se mensuráveis com precisão.
- **Iterações e reprodutibilidade:** antes de medir, cada cenário executa repetições de *aquecimento* (descartadas) para que o compilador JIT do V8 atinja regime estável. Em seguida, o tempo é medido em 10 janelas (cada uma com K repetições, conforme acima); reportam-se a **média aritmética** por unidade de trabalho e o respectivo desvio-padrão (coluna `timeMsStd` do arquivo `benchmark_results.csv`). A geração procedural de obstáculos utiliza PRNG determinístico (mulberry32) com sementes fixas por suíte, de modo que as métricas estruturais — comprimento do caminho (`pathLength`) e nós expandidos (`nodesVisited`) — são idênticas em qualquer execução e em qualquer máquina. Já o tempo (`timeMs`) permanece sujeito a variações do motor V8 (compilação JIT, coletor de lixo, concorrência com o sistema operacional) e, portanto, não pretende ser generalizável a outros hardwares; com a medição em lote, contudo, o desvio-padrão observado ficou abaixo de 0,01 ms nas buscas isoladas e abaixo de 0,07 ms nos cenários com múltiplos NPCs.
- **Isolamento:** cada suíte cria uma instância de `Grid` e `PathfindingSystem`; os campos de busca das células são reinicializados antes de cada chamada (`resetPathfindingData`), de modo que o resultado de um algoritmo não contamina o de outro. A contagem de nós expandidos é instrumentada *dentro* do `PathfindingSystem`, de forma idêntica para todos os algoritmos: nenhuma medição de tempo é penalizada por instrumentação (como a reescrita de `getNeighbors`) que os demais algoritmos não sofram.

Definição da métrica `nodesVisited`: para garantir comparabilidade com a literatura clássica de busca informada, a métrica `nodesVisited` reportada neste trabalho contabiliza exclusivamente *nós expandidos* — isto é, nós extraídos da lista aberta durante o laço principal de cada algoritmo (*extract-min* no A*, Dijkstra e JPS; *dequeue* no BFS; *pop* no DFS). No caso específico do JPS, isso significa contar apenas os *jump points* efetivamente inseridos e posteriormente extraídos da fila de prioridade, *não* as células percorridas pelas sondagens internas da rotina `jump()`. Essa escolha alinha a métrica ao conceito de “nodes expanded” usado por Harabor and Grastien (2011) e torna válida a comparação direta entre A* e JPS — esperando-se redução expressiva do contador pela poda de simetrias, não aumento causado por varreduras auxiliares.

O simulador e o código-fonte estão disponíveis publicamente no repositório GitHub do projeto: <https://github.com/jeandeson/pathfinding-quest>. Uma versão hospedada está acessível em <https://pathfinding-quest-j95b.vercel.app/>, permitindo que a banca avaliadora execute os benchmarks e visualize os algoritmos diretamente no navegador, sem necessidade de instalação.

3.3. Modelagem do Ambiente

A forma de representar o ambiente do jogo é de suma importância nos quesitos de desempenho e exatidão do algoritmo. Por esse motivo, essa metodologia utiliza o uso de matriz, pois traz um ótimo equilíbrio entre o simples e o flexível. Essa estrutura mínima é uma matriz bidimensional onde a célula é navegável ou não. A Figura 1 ilustra essa representação no simulador em execução, no modo de depuração.

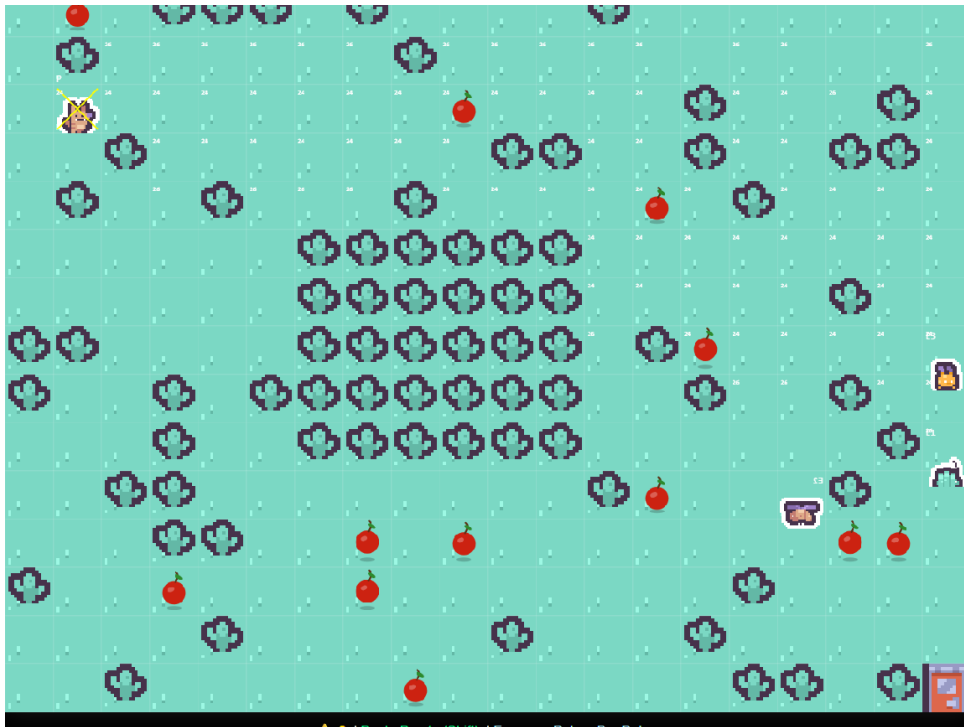


Figura 1. Simulador interativo desenvolvido, em modo de depuração: a grade de *pathfinding* e os custos das células são sobrepostos ao mapa do jogo *top-down*.

A interface `Cell` (Apêndice A, Listagem A.1) define os atributos de cada célula da grade: coordenadas lógicas (`x`, `y`), posição no mundo (`worldX`, `worldY`), flag de navegabilidade (`walkable`), ponteiro para o nó pai na reconstrução do caminho, e os valores f , g e h utilizados pelos algoritmos informados. A classe `Grid` (Apêndice A, Listagem A.2) encapsula a matriz e disponibiliza os métodos `getNeighbors` — que retorna células adjacentes navegáveis — e `findPath` — que delega ao algoritmo selecionado. Obstáculos são gerados proceduralmente via PRNG com semente determinística, garantindo reprodutibilidade dos testes (Seção 3.2). Essa estrutura permite representar de maneira fiel um ambiente 2D, tornando fácil integrar o A* com agentes e o loop indispensável na classe `Game`.

3.4. Implementação do Algoritmo Dijkstra

A implementação é baseada no trabalho de Dijkstra (1959) e aprofundada em Cormen et al. (2009). O algoritmo mantém uma lista aberta inicializada com o nó de partida e itera selecionando o nó de menor custo $g(n)$ acumulado. Diferentemente do A*, não utiliza heurística: todo o espaço de busca alcançável é explorado até o objetivo ser atingido, garantindo otimalidade ao custo de maior número de nós visitados. A lista aberta é implementada como um **heap binário** (fila de prioridade mínima, Listagem A.9), de modo que cada extração do nó de menor custo executa em $O(\log V)$ e o laço principal em $O((V + E) \log V)$. O código completo está no Apêndice A (Listagem A.3).

3.5. Implementação do Algoritmo de busca em largura BFS

O BFS, conforme estudado em Rahim et al. (2018), utiliza uma fila FIFO e garante o caminho de menor número de passos em grades com custo uniforme. Cada nó é visitado exatamente uma vez: quando removido da fila, seus vizinhos não visitados são enfileirados com referência ao nó pai para reconstrução do caminho. A complexidade é $O(V + E)$ em número de nós e arestas. O código completo está no Apêndice A (Listagem A.4).

3.6. Implementação do Algoritmo de busca em Profundidade DFS

A implementação adota a variante *Greedy DFS* descrita na Seção 2.4: os vizinhos são ordenados por distância Manhattan decrescente antes de serem empilhados, de modo que o vizinho mais próximo do objetivo fique no topo da pilha (LIFO) e seja expandido primeiro. O rastreamento do caminho utiliza um mapa auxiliar `cameFrom` em vez do campo `parent` da célula, evitando corrupção entre ramos distintos da busca. Um limite de profundidade igual ao total de células da grade (`MAX_DEPTH`) atua como salvaguarda. Vale observar que esta implementação mantém um conjunto `visited` e um mapa `cameFrom`, ambos $O(V)$; portanto, ao contrário do DFS recursivo canônico — cujo consumo de memória é $O(\text{profundidade})$ —, a variante aqui adotada **não preserva a vantagem clássica de memória do DFS**, trocando-a por prevenção de ciclos e reconstrução de caminho robusta. Como a pilha é explícita (e não recursiva), também não há risco de estouro da pilha de chamadas. O código completo está no Apêndice A (Listagem A.5).

3.7. Implementação do Algoritmo A*

A metodologia apresenta uma forma de implementar o A de maneira modular, tornando simples para adaptar ou mudar caso haja necessidade. Os componentes essenciais incluem um método na Classe `Grid` chamada de `pathFinder` que encapsula a lógica principal do algoritmo e recebe a célula inicial, a final e o algoritmo que está sendo utilizado.

Isso permite trocar o algoritmo facilmente, apenas passando diferentes parâmetros para o método.

A forma de implementar o A* segue o algoritmo básico, inicializando os conjuntos aberto e fechado, iterando e selecionando o nó com menor valor $f(n)$ para expandir e reconstruir o caminho depois de encontrar o objetivo. Em pseudocódigo, o laço principal pode ser descrito como:

1. Inicializar *openSet* com o nó de partida e *closedSet* vazio; atribuir $g = 0$, $h = \text{heuristic}(\text{start}, \text{goal})$ e $f = g + h$ ao nó inicial.
2. Enquanto *openSet* não estiver vazio: selecionar o nó *current* com menor f ; se *current* for o objetivo, reconstruir e retornar o caminho pelos ponteiros *parent*.
3. Caso contrário, mover *current* para *closedSet* e, para cada vizinho transitável: calcular $g_{\text{tentativo}} = g(\text{current}) + 1$; se o vizinho estiver fora de *openSet* ou o novo g for menor, atualizar *parent*, g , h e $f = g + h$.
4. Se o laço terminar sem atingir o objetivo, retornar `null` (sem caminho).

Assim como no Dijkstra, a lista aberta (*openSet*) é um **heap binário**: a seleção do nó de menor f custa $O(\log V)$, resultando em complexidade $O((V + E) \log V)$ — assintoticamente superior a uma *open list* em vetor com varredura linear, cujo *extract-min* e teste de pertinência seriam $O(V)$, degradando o laço para $O(V^2)$. O código TypeScript completo do método está no Apêndice A (Listagem A.6).

3.8. Implementação do Jump Point Search (JPS)

A implementação do JPS segue a proposta de Harabor and Grastien (2011) adaptada para movimentos 4-direcionais (cardeais). O algoritmo mantém a mesma estrutura de lista aberta e fechada do A*, diferindo na forma como os sucessores são identificados: em vez de expandir todos os vizinhos adjacentes, o JPS aplica a função *jump* que percorre uma linha reta até encontrar um ponto de salto ou um obstáculo.

Em pseudocódigo, a função *jump*, invocada a partir de uma célula (x, y) e uma direção $\vec{d} = (dx, dy)$, procede assim: avança uma célula por vez na direção $\vec{d}$ enquanto a célula corrente for válida e transitável, retornando a célula se ela for o objetivo ou se possuir vizinhos forçados (a contagem de nós expandidos, conforme a Seção 3.2, registra apenas os *jump points* extraídos da fila de prioridade, não as células percorridas durante esta varredura). Em grades 4-direcionais, também realiza uma *sondagem perpendicular* linear em cada direção ortogonal a $\vec{d}$: se a sondagem encontrar um ponto de salto, a célula corrente também é um ponto de salto. Se ocorrer um obstáculo ou a célula sair da grade, a função retorna `null`. O código TypeScript completo está no Apêndice A (Listagem A.7).

A sondagem perpendicular (*jpsProbe*) percorre uma linha reta a partir do nó corrente em cada direção ortogonal ao movimento principal, retornando `true` se encontrar o objetivo ou uma célula com vizinho forçado. Essa etapa é necessária na adaptação 4-direcional porque, sem ela, pontos de curva em corredores sem obstáculos próximos nunca seriam identificados como *jump points*, levando o algoritmo a ignorar caminhos válidos. Como a sondagem é executada a cada célula percorrida pela rotina de salto, ela introduz um custo constante por passo que não existe no JPS 8-direcional clássico; esse custo é determinante para o comportamento de tempo discutido na Seção 4, onde a vantagem do JPS em *tempo* (e não apenas em nós expandidos) só se manifesta quando a densidade de obstáculos é suficientemente alta para encurtar as varreduras.

A detecção de vizinhos forçados verifica se há um obstáculo na célula imediatamente “atrás e ao lado” do nó atual — indicando que o caminho ótimo até o vizinho lateral não pode ser feito sem passar pelo nó corrente. Em pseudocódigo, para um movimento horizontal ($dx \neq 0$), verifica se existem células transitáveis acima ou abaixo da célula corrente cujo “vizinho de trás” (na direção $-dx$) seja um obstáculo; analogamente, para movimento vertical ($dy \neq 0$), verifica vizinhos à esquerda e à direita. Caso qualquer uma dessas condições seja satisfeita, o nó corrente é um ponto de salto. O código completo está no Apêndice A (Listagem A.8).

Como os pontos de salto não são necessariamente adjacentes, o caminho retornado pelo JPS interpola as células intermediárias entre *jump points* consecutivos, garantindo que os NPCs possam se mover célula a célula normalmente.

4. Resultados e Discussão

Nesta seção são apresentados os resultados obtidos a partir dos testes realizados com os cinco algoritmos de pathfinding implementados (A*, Dijkstra, BFS, DFS e JPS). Os testes foram conduzidos em cenários simulados com grades de diferentes tamanhos e densidades de obstáculos, medindo métricas de comprimento do caminho, nós visitados, tempo de execução (ms) e taxa de sucesso. Para garantir reprodutibilidade, cada configuração de teste utiliza sementes determinísticas fixas na geração de obstáculos, de modo que as métricas estruturais (comprimento do caminho e nós visitados) são idênticas em qualquer execução do simulador. Os tempos de execução reportados são médias de 10 janelas cronometradas em lote, em Chrome *headless*, conforme descrito na Seção 3.2 — metodologia que mantém as medidas acima do piso de resolução do relógio do navegador e elimina o viés de instrumentação entre algoritmos.

4.1. Casos de Teste

Os casos de teste foram projetados para validar a funcionalidade e eficiência dos algoritmos em cenários variados, simulando condições reais de jogos 2D.

4.2. Mapas com Diferentes Configurações de Obstáculo

Nos testes com grade 20×20 e densidade de obstáculos de 0,2, os algoritmos A*, Dijkstra, BFS e JPS encontraram caminhos ótimos com comprimento de 39 células. O DFS (*Greedy DFS*) encontrou caminho de 45 células — válido mas subótimo, 15% mais longo que o ótimo, comportamento esperado para uma busca sem garantia de optimalidade. O JPS expande consistentemente menos nós da lista aberta do que o A* em todas as densidades testadas, coerente com a análise de Harabor and Grastien (2011): a poda de simetrias e o salto até *jump points* evitam a inserção de vizinhos intermediários na fila de prioridade. Em densidade 0,2 o JPS expande 66 nós contra 103 do A*, e em densidade 0,4 a vantagem se amplia drasticamente — apenas 3 nós contra 399 do A*, pois obstáculos abundantes criam vizinhos forçados que encurtam radicalmente as varreduras.

Vale analisar a relação entre nós expandidos e tempo. Na densidade 0,2, o A* expande bem menos nós que o BFS (103 contra 284) e, com a *open list* em heap binário, também é mais rápido (0,01 ms contra 0,02 ms): a heurística de Manhattan guia a busca para perto do objetivo, enquanto o BFS explora uniformemente por níveis. Essa vantagem de tempo do A* sobre o BFS, contudo, não é universal — em grades muito esparsas o

Tabela 1. Resultados para Mapas com Obstáculos (Grade 20×20, Densidade 0,2)

Algoritmo	Comprimento	Nós Visitados	Tempo Médio (ms)	Taxa Sucesso
A*	39	103	0,01	100%
Dijkstra	39	284	0,03	100%
BFS	39	284	0,02	100%
DFS	45	50	0,01	100%
JPS	39	66	0,02	100%

custo por nó do A* (cálculo de h e manutenção do heap) pode superar a economia de expansões, e o BFS, com sua fila FIFO de custo $O(1)$, torna-se competitivo ou até mais rápido (Seção 4.3). O JPS expande ainda menos nós (66), mas seu tempo (0,02 ms) é ligeiramente superior ao do A* nesta grade pequena e esparsa: o custo das sondagens perpendiculares realizadas a cada célula da rotina `jump()` não é amortizado por tão poucas expansões. Como mostram as próximas seções, a vantagem do JPS em *tempo* só se manifesta quando a densidade de obstáculos cresce, encurtando as varreduras (Tabela 2).

A Tabela 2 apresenta os resultados para a densidade 0,4, onde a vantagem do JPS em nós visitados torna-se mais evidente.

Tabela 2. Resultados para Mapas com Obstáculos (Grade 20×20, Densidade 0,4)

Algoritmo	Comprimento	Nós Visitados	Tempo Médio (ms)	Taxa Sucesso
A*	39	399	0,05	100%
Dijkstra	39	399	0,05	100%
BFS	39	399	0,03	100%
DFS	39	38	0,01	100%
JPS	39	3	0,02	100%

Em densidade 0,4, o JPS expande apenas 3 nós contra 399 do A* — redução de mais de 99%. Obstáculos mais densos criam mais vizinhos forçados, que são exatamente os pontos que o JPS identifica para fazer curvas: com o caminho viável fortemente restringido pelo ambiente, poucos *jump points* são suficientes para conectar origem e destino, eliminando quase integralmente a necessidade de expandir nós intermediários. O DFS (*Greedy DFS*) encontrou o caminho ótimo de 39 células com apenas 38 expansões nessa densidade — fenômeno explicado pelo estreitamento das rotas viáveis: com muitos obstáculos, a heurística guia o DFS diretamente pela única sequência transitável, sem necessidade de backtracking, convergindo ao mesmo caminho que os algoritmos ótimos. A Figura 2 consolida graficamente os nós visitados nas duas densidades.

4.3. Testes de Desempenho com Múltiplos NPCs

Para 50 NPCs simultâneos em grade 20×20 com densidade 0,2, todos os algoritmos ótimos (A*, Dijkstra, BFS, JPS) mantiveram caminhos de comprimento 39, e o tempo total escalou linearmente com o número de NPCs, já que cada NPC executa uma busca independente. Todos os algoritmos completaram as 50 buscas em menos de 2 ms — folgadoamente dentro do orçamento de 16 ms de um quadro a 60 fps. O *Greedy DFS* foi o mais rápido (0,51 ms), por expandir o menor número de nós, seguido de perto por A* (0,95 ms) e BFS (0,96 ms); o JPS (1,06 ms) e o Dijkstra (1,62 ms) foram os mais



Figura 2. Nós visitados por algoritmo em grade 20x20, nas densidades 0,2 e 0,4 — dados das Tabelas 1 e 2. O JPS reduz mais de 99% das expansões em alta densidade.

lentos. Observa-se que, embora o JPS expanda muito menos nós que o A* (3.500 contra 7.100), *não* é o mais rápido em tempo nesta grade esparsa: o custo fixo das sondagens perpendiculares por salto supera a economia de inserções na fila de prioridade. O DFS, embora rápido, produziu caminhos de 45 células — subótimos frente ao ótimo de 39. Os tempos exatos são apresentados na Tabela 3 e ilustrados na Figura 3.

Tabela 3. Resultados para Múltiplos NPCs (Grade 20x20, Densidade 0,2, 50 NPCs)

Algoritmo	Compr.	Nós Visitados (total)	Tempo Total Médio (ms)	Taxa Sucesso
A*	39	7.100	0,95	100%
Dijkstra	39	14.700	1,62	100%
BFS	39	14.700	0,96	100%
DFS	45	2.350	0,51	100%
JPS	39	3.500	1,06	100%

4.4. Teste de Escalabilidade

Os dados confirmam que o tempo cresce linearmente com o número de NPCs para todos os algoritmos, dado que cada NPC executa uma busca independente. Em grades 40x40, o A* visita 487 nós (densidade 0,2) e 1.599 nós (densidade 0,4), evidenciando que o espaço de busca cresce com a grade e com a presença de obstáculos que desviam o caminho. O JPS mantém a tendência de redução expressiva de nós em densidades elevadas (apenas 3 nós para 40x40, densidade 0,4, contra 1.599 do A*), confirmando que a poda direcional é mais eficaz quando obstáculos criam vizinhos forçados frequentes.

4.5. Teste de Congestionamento

Nos testes de congestionamento em grade 30x30 com corredor central forçado, todos os algoritmos obtiveram sucesso apenas na densidade 0,2. Nas densidades 0,4 e

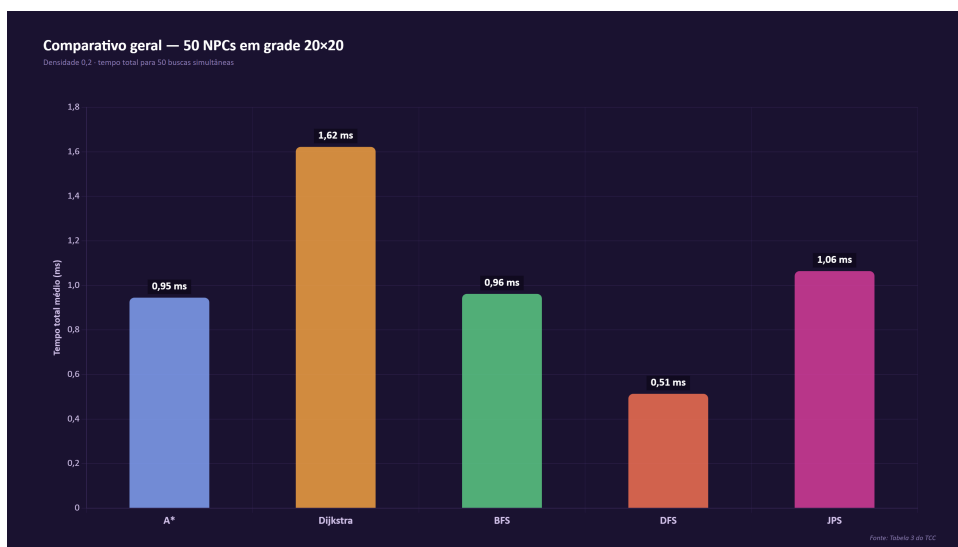


Figura 3. Tempo total para 50 buscas simultâneas em grade 20×20, densidade 0,2 — dados da Tabela 3. Todos os algoritmos permanecem muito abaixo do orçamento de 16 ms de um quadro a 60 fps (linha tracejada).

0,6, o corredor ficou completamente bloqueado e todos os algoritmos retornaram falha — comportamento correto, indicando ausência de caminho transitável. Na densidade 0,2, todos os algoritmos encontraram o caminho ótimo de 46 células. O DFS (*Greedy DFS*) também obteve 46 células nesse cenário: a estrutura de corredor forçado deixa poucas opções de desvio, de modo que a ordenação heurística conduz o DFS pelo mesmo trajeto direto que os demais algoritmos, visitando apenas 75 nós — menos que BFS e Dijkstra (375 cada), como detalha a Tabela 4.

Tabela 4. Resultados para Congestionamento (Grade 30×30, Densidade 0,2 — única com caminho possível)

Algoritmo	Comprimento	Nós Visitados	Tempo Médio (ms)	Taxa Sucesso
A*	46	200	0,02	100%
Dijkstra	46	375	0,04	100%
BFS	46	375	0,03	100%
DFS	46	75	0,01	100%
JPS	46	127	0,03	100%
Densidades 0,4 e 0,6: todos os algoritmos falharam (corredor bloqueado)				

Um aspecto relevante nesse cenário é que, em JavaScript, cada NPC executa uma busca *independente* — não há compartilhamento de estrutura de dados entre agentes, pois o motor é *single-thread* e as buscas são sequenciais dentro do mesmo quadro. Quando N NPCs convergem para o mesmo corredor, o custo total escala linearmente: $N \times \text{custo_individual}$. Para validar essa projeção com dados reais, o benchmark de congestionamento foi executado com 5, 10, 20 e 50 NPCs na densidade 0,2 (única com caminho válido). Os resultados confirmam a escala linear e são apresentados na Tabela 5 e ilustrados nas Figuras 4 e 5. Com 50 NPCs, *todos* os algoritmos permanecem confortavelmente dentro do orçamento de 16 ms de um quadro a 60 fps: o DFS é o mais rápido (0,71 ms), seguido por BFS (1,25 ms), A* (1,28 ms), JPS (1,71 ms) e Dijkstra (2,05 ms).

Ainda assim, a escala linear permite projetar o ponto de saturação: como cada busca A* custa cerca de 0,026 ms neste corredor, o orçamento de 16 ms só seria atingido na casa das centenas de NPCs convergentes. Em jogos que se aproximem dessa ordem de grandeza — ou em grades muito maiores —, distribuir os replanejamentos ao longo de múltiplos quadros (replanejando alguns NPCs por quadro em vez de todos) é a estratégia prática para preservar o orçamento de tempo real.

Tabela 5. Congestionamento com Múltiplos NPCs (Grade 30×30, Densidade 0,2, corredor central)

Algoritmo	NPCs	Nós Visitados (total)	Tempo Médio (ms)	Taxa Sucesso
A*	5	1.000	0,12	100%
A*	10	2.000	0,24	100%
A*	20	4.000	0,48	100%
A*	50	10.000	1,28	100%
Dijkstra	5	1.875	0,21	100%
Dijkstra	10	3.750	0,42	100%
Dijkstra	20	7.500	0,82	100%
Dijkstra	50	18.750	2,05	100%
BFS	5	1.875	0,13	100%
BFS	10	3.750	0,25	100%
BFS	20	7.500	0,50	100%
BFS	50	18.750	1,25	100%
DFS	5	375	0,07	100%
DFS	10	750	0,14	100%
DFS	20	1.500	0,28	100%
DFS	50	3.750	0,71	100%
JPS	5	635	0,17	100%
JPS	10	1.270	0,34	100%
JPS	20	2.540	0,68	100%
JPS	50	6.350	1,71	100%

4.6. Teste de Objetivos Dinâmicos

Nos testes de objetivos dinâmicos em grade 40×40 com densidade 0,25, A*, Dijkstra e JPS obtiveram sucesso nas Etapas 1 e 3, mas falharam na Etapa 2 (destino na coordenada 20,20 — região central isolada pela semente de obstáculos). Essa falha é consistente entre todos os algoritmos testados, confirmando que a célula-destino estava genuinamente inacessível nessa configuração de mapa, não sendo um erro de implementação. A Etapa 3 (destino próximo à borda) foi bem-sucedida em todos os algoritmos, e os comprimentos de caminho são idênticos (81 e 59 células), confirmando otimalidade equivalente.

Este resultado evidencia uma limitação importante em ambientes dinâmicos com replanejamento sequencial: a ausência de detecção de conectividade prévia pode levar a buscas exaustivas sem caminho. Uma mitigação imediata seria verificar a conectividade via BFS antes de iniciar a busca principal — ao custo de uma varredura adicional de $O(V)$, elimina-se o desperdício da busca exaustiva observado na Etapa 2.

Além disso, o replanejamento sequencial impõe um custo acumulado que merece análise quantitativa. Em um jogo a 60 fps, cada quadro dispõe de aproximadamente

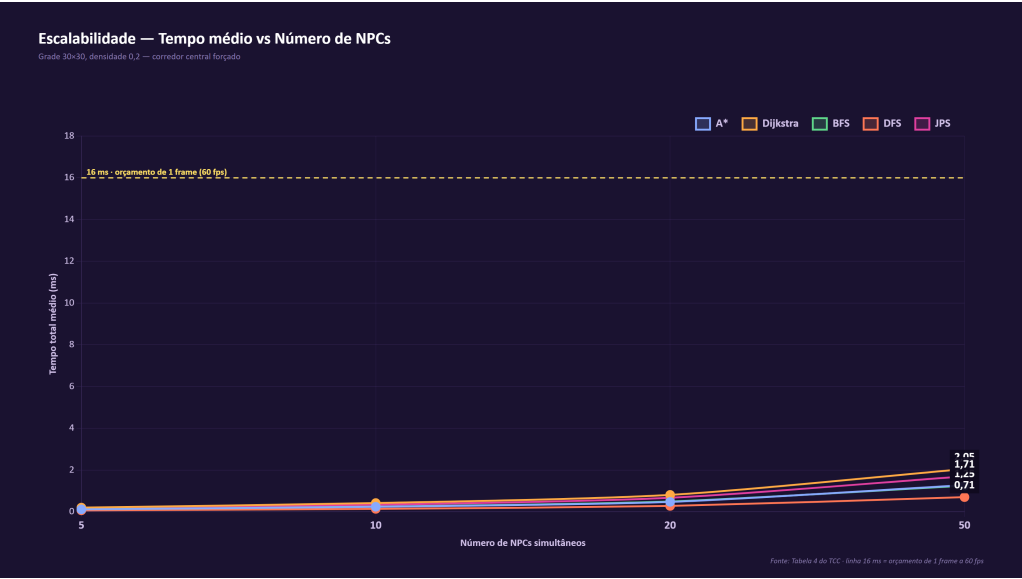


Figura 4. Escalabilidade do tempo total médio com o número de NPCs no corredor 30×30, densidade 0,2 — dados da Tabela 5. A linha tracejada marca o orçamento de 16 ms (60 fps).

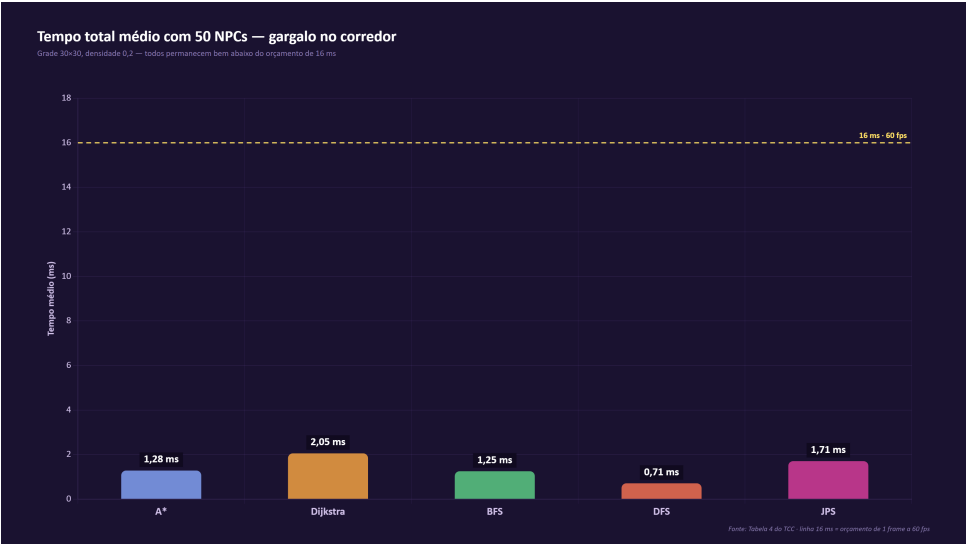


Figura 5. Tempo total médio com 50 NPCs no corredor 30×30, densidade 0,2 — dados da Tabela 5. Todos os algoritmos permanecem bem abaixo do orçamento de 16 ms (60 fps); o gráfico evidencia o espalhamento relativo entre eles.

16,67 ms de orçamento, dos quais a *pathfinding* disputa tempo com renderização, lógica de jogo, áudio e processamento de *input*. No cenário mais exigente medido — 50 NPCs convergindo para um corredor central (30×30, densidade 0,2, Tabela 5) — o custo por quadro em relação aos 16,67 ms é pequeno em todos os casos: DFS 0,71 ms ($\approx 4\%$), BFS 1,25 ms ($\approx 8\%$), A* 1,28 ms ($\approx 8\%$), JPS 1,71 ms ($\approx 10\%$) e Dijkstra 2,05 ms ($\approx 12\%$). Ou seja, nas dimensões testadas, *todos* os algoritmos deixam ampla margem para os demais subsistemas; o orçamento de 16 ms só se torna restritivo ao projetar para grades muito maiores ou contagens de agentes uma ordem de grandeza acima das aqui avaliadas.

Em ambientes *verdadeiramente dinâmicos*, onde obstáculos móveis obrigam os agentes a replanejar a *cada quadro*, esse custo se multiplica: para 60 fps e 50 NPCs, são 3.000 replanejamentos por segundo. Projetando para grades maiores e mais densas, cada busca A* em 40×40 com densidade 0,4 custa cerca de 0,24 ms (Seção 4.1); com 50 NPCs replanejando a cada quadro, isso implicaria ≈ 12 ms/*quadro* apenas para a *pathfinding* — ainda dentro do orçamento de 16 ms, porém consumindo a maior parte dele e deixando pouca folga para renderização e lógica. O **corolário prático** é que, embora o A* puro seja viável nas escalas testadas, seu uso em ambientes com obstáculos móveis e dezenas de agentes em grades grandes se aproxima do limite do quadro; nesses casos convém recorrer a substituição algorítmica (D* Lite), amortização temporal (replanejar parte dos NPCs por quadro) ou redução de escopo (busca hierárquica, HPA*).

O algoritmo D* Lite Koenig and Likhachev (2002), projetado para replanejamento incremental, propaga apenas as alterações causadas por mudanças no grafo, com custo tipicamente proporcional ao número de células afetadas pela mudança — em geral muito menor que o total de células V — em vez de recomputar toda a busca. Em mudanças locais (um obstáculo que aparece ou desaparece em uma célula), k pode ser da ordem de poucas dezenas mesmo em grades de milhares de células, reduzindo em uma ou duas ordens de grandeza o custo por replanejamento em relação à recomputação completa do A*. Em ambientes onde os obstáculos mudam com frequência, o D* Lite é, portanto, a direção natural para trabalhos futuros neste contexto web — especialmente se combinado com amortização temporal para distribuir o custo residual entre *frames*.

Tabela 6. Resultados para Objetivos Dinâmicos (Grade 40×40, Densidade 0,25)

Algoritmo	Etapas	Destino	Comprimento	Nós Visitados	Tempo Médio (ms)
A*	1	(39,39)	81	408	0,05
A*	2	(20,20)	0	1051	0,18
A*	3	(5,35)	59	335	0,04
Dijkstra	1	(39,39)	81	1050	0,17
Dijkstra	2	(20,20)	0	1051	0,17
Dijkstra	3	(5,35)	59	774	0,11
JPS	1	(39,39)	81	266	0,07
JPS	2	(20,20)	0	708	0,23
JPS	3	(5,35)	59	216	0,05

4.7. Análise de Eficiência

A análise revela *trade-offs* distintos entre os algoritmos. O JPS destaca-se pelo **menor número de nós expandidos** em todas as densidades testadas — vantagem mo-

desta em grades esparsas (66 contra 103 do A* na densidade 0,2) e dramática em grades densas (3 contra 399 do A* na densidade 0,4), onde vizinhos forçados mais frequentes encurtam radicalmente as varreduras. Essa economia de expansões, contudo, nem sempre se traduz em menor **tempo**: em grades pequenas e esparsas, o custo das sondagens perpendiculares torna o JPS mais lento que o A* (e até que o BFS), e sua vantagem temporal só emerge em alta densidade. O DFS, por expandir pouquíssimos nós, registra o menor tempo em vários cenários, mas não garante otimalidade do caminho.

4.8. Desempenho Computacional

Em termos de nós expandidos, o JPS apresenta vantagem sistemática: na densidade 0,2 (grade 20×20) expande 66 nós contra 103 do A* e 284 do BFS/Dijkstra; na densidade 0,4, expande apenas 3 contra 399 do A*. Em tempo, na densidade 0,2 (20×20) o A* é o mais rápido entre os algoritmos baseados em fila de prioridade (0,01 ms), seguido do JPS (0,02 ms); o Dijkstra é o mais lento entre eles (0,03 ms). É importante notar que o Dijkstra é mais lento que o BFS *apesar de expandir exatamente o mesmo número de nós* (284): a diferença não decorre da ausência de heurística — o BFS também não a utiliza — mas do custo de extração da fila de prioridade em heap ($O(\log n)$) frente ao *dequeue* de custo $O(1)$ da fila FIFO do BFS. Para 50 NPCs, todos os tempos ficam entre 0,5 e 1,6 ms; o DFS é o mais rápido (por expandir menos nós) e o JPS, ao contrário do que a contagem de nós poderia sugerir, *não* é o mais rápido.

4.9. Qualidade dos Caminhos

A*, Dijkstra, BFS e JPS garantem caminhos ótimos (comprimento médio 39 células em grades 20×20), pois todos utilizam heurísticas admissíveis ou exploração uniforme por nível. O DFS (variante *Greedy DFS*) não possui essa garantia: nos testes com grade 20×20 e densidade 0,2, encontrou caminhos de 45 células — 15% mais longos que o ótimo de 39. Em densidade 0,4 e no teste de corredor (congestionamento), o DFS encontrou o caminho ótimo: a alta densidade de obstáculos restringe as rotas viáveis, e a ordenação heurística conduz o algoritmo pelo mesmo trajeto que os demais, sem necessidade de backtracking. Esse padrão revela que a subotimalidade do DFS é mais pronunciada em ambientes abertos (baixa densidade) e diminui conforme o espaço de busca se torna mais restrito. Esse comportamento é intrínseco à estrutura do DFS: a orientação heurística direciona a busca, mas não impede que o algoritmo comprometa a otimalidade ao descer profundamente por um ramo antes de considerar alternativas mais curtas. Para NPCs em jogos, caminhos subótimos são toleráveis quando a velocidade de cálculo é prioritária; para aplicações que exigem o menor trajeto, A*, BFS ou JPS são necessários. Em cenários de alta densidade de obstáculos (0,6), todos os algoritmos falham quando o destino está isolado — comportamento correto, indicando ausência de caminho.

4.10. Análise do DFS: Velocidade versus Qualidade de Caminho

O *Greedy DFS* apresenta o menor número de nós visitados entre todos os algoritmos testados e tempos de execução inferiores ao A* em grades pequenas. Esse desempenho, porém, é consequência direta de um compromisso: ao descer profundamente pelo ramo mais promissor sem retroceder antecipadamente, o algoritmo encontra *uma* solução rapidamente, mas não *a melhor* solução. Os caminhos produzidos são sistematicamente mais longos que o ótimo em todos os cenários testados.

Do ponto de vista prático, o DFS puro — sem ordenação heurística — revelou-se inaplicável como algoritmo de navegação de agentes em jogos 2D em tempo real. Em testes realizados durante o desenvolvimento, o DFS puro gerava rotas que percorriam regiões opostas ao objetivo antes de redirecionar, tornando o comportamento dos NPCs visivelmente incorreto para o jogador. Isso ilustra por que o DFS canônico não é adequado para *pathfinding* quando a qualidade do caminho importa: ele não oferece garantia de caminho mínimo Cormen et al. (2009). A variante *Greedy DFS* adotada neste trabalho resolve o problema de usabilidade em jogo ao custo de abrir mão da otimalidade, tornando-se comparável aos demais algoritmos no contexto de NPCs onde velocidade de replanejamento é mais crítica que o comprimento exato da rota.

5. Conclusão

Este trabalho analisou comparativamente cinco algoritmos de *pathfinding* — A*, Dijkstra, BFS, DFS e Jump Point Search (JPS) — implementados em JavaScript/TypeScript sobre a API Canvas do navegador, sem dependências externas. Os experimentos responderam às hipóteses levantadas na introdução:

Sobre o orçamento de tempo real (16ms por quadro): A estimativa inicial de 50ms por busca, adotada no anteprojeto, revelou-se folgada em mais de uma ordem de grandeza: uma busca isolada em grade 20×20 com densidade 0,2 custa apenas centésimos de milissegundo, pois percorre poucas centenas de nós. O limiar verdadeiramente relevante para jogos, portanto, não é o custo de uma busca isolada, mas o **orçamento de aproximadamente 16ms de um quadro a 60fps** — compartilhado por todas as buscas daquele quadro, além de renderização, lógica e *input*. Avaliado por esse critério, com a *open list* em heap binário e medição livre de viés de instrumentação, o resultado é claro: com 50 NPCs simultâneos em grade 20×20, *todos* os cinco algoritmos completaram as buscas em menos de 2 ms — DFS 0,51, A* 0,95, BFS 0,96, JPS 1,06 e Dijkstra 1,62 ms —, permanecendo folgadamente dentro do orçamento. Nas escalas testadas, portanto, a escolha do algoritmo não é ditada pela viabilidade de *frame rate* (todos são viáveis), mas pelo número de nós expandidos e pela qualidade do caminho; pela escala linear medida, o orçamento de 16ms só se torna fator limitante ao projetar para grades muito maiores ou para uma ordem de grandeza a mais de agentes.

Sobre a relação entre A* e BFS: O A* expande sistematicamente menos nós que o BFS (103 contra 284 na densidade 0,2; 1.154 contra 1.299 em 40×40, densidade 0,1), pois a heurística o direciona ao objetivo. Em *tempo*, contudo, os dois são próximos e nenhum domina universalmente: na densidade 0,2 (20×20) o A* é mais rápido (0,01 contra 0,02 ms), mas em grades esparsas o custo por nó do A* (heurística e heap) pode anular a economia de expansões, e o BFS — com sua fila FIFO de custo $O(1)$ — chega a ser mais rápido (por exemplo, em 40×40 com densidade 0,1). A conclusão honesta é que a vantagem do A* sobre o BFS é *clara em nós expandidos* e *condicional em tempo*, dependendo de quão densa é a grade e de quanto a heurística consegue podar.

Sobre o JPS como otimização do A*: O JPS demonstrou vantagem *sistemática em nós expandidos*: expandiu menos nós que o A* em todas as densidades, com redução superior a 99% em grades densas (3 contra 399 na densidade 0,4). Essa economia, porém, *nem sempre se converte em menor tempo*. Em grades pequenas e esparsas, o custo das sondagens perpendiculares executadas a cada célula da rotina de salto torna o JPS mais

lento que o A* (por exemplo, 50 NPCs em 20×20: JPS 1,06 ms contra 0,95 ms do A*); sua vantagem em *tempo* só se manifesta quando a densidade de obstáculos é alta o bastante para encurtar as varreduras (40×40, densidade 0,4: 0,06 ms do JPS contra 0,24 ms do A*). Em todos os casos testados, o JPS encontrou caminhos de comprimento idêntico ao A* e ao BFS, validando empiricamente a otimalidade da variante 4-direcional adotada. O JPS é, portanto, indicado quando o gargalo é o *número de expansões* (grades grandes e densas), não necessariamente quando o gargalo é o tempo por busca em grades pequenas.

Sobre o DFS e o tradeoff velocidade–qualidade: O DFS, implementado na variante *Greedy DFS* com ordenação heurística dos vizinhos, visitou consistentemente menos nós que A*, BFS e Dijkstra, apresentando os menores tempos em grades isoladas. Contudo, produziu caminhos sistematicamente subótimos (até 15% mais longos que o ótimo). Testes com DFS puro — sem ordenação — demonstraram que o algoritmo canônico é inaplicável para agentes de jogo em tempo real: sem direcionamento heurístico, o DFS percorre regiões opostas ao objetivo antes de redirecionar, gerando comportamento incoerente nos NPCs. Isso evidencia que velocidade de busca e qualidade de caminho são métricas conflitantes, e que a escolha do algoritmo deve ser guiada pela prioridade do sistema: para máxima velocidade com qualidade tolerável, *Greedy DFS*; para caminho ótimo com boa performance, A* ou JPS.

Sobre ambientes dinâmicos: A falha do A* e do JPS na Etapa 2 do teste dinâmico não representou uma limitação dos algoritmos em si, mas uma propriedade do mapa gerado — a célula-destino estava isolada por obstáculos. Ambos os algoritmos responderam corretamente ao retornar `null` após explorar o espaço alcançável. O custo desse replanejamento malsucedido evidencia a necessidade de verificação de conectividade prévia em ambientes dinâmicos, evitando varreduras exaustivas desnecessárias. Em um cenário de jogo real com 50 NPCs replanejando continuamente a 60fps, o custo acumulado medido (Seção 4.3) representou uma fração pequena do orçamento de 16ms por quadro (cerca de 1 a 2 ms, ou 4–12%), com ampla folga nas dimensões testadas; o custo só se tornaria restritivo ao projetar para grades muito maiores ou replanejamento a cada quadro. Para esses casos-limite, o D* Lite Koenig and Likhachev (2002) oferece replanejamento incremental cujo custo é proporcional ao número de células afetadas pela mudança — substancialmente mais eficiente que recomputar todo o A* a cada atualização do ambiente.

A metodologia em JavaScript/TypeScript com Canvas mostrou-se adequada para a análise proposta: a ausência de frameworks de jogo elimina variáveis de desempenho externas, e a execução no navegador permite reprodutibilidade imediata. O simulador funcional entregue como artefato do curso permite que qualquer avaliador verifique os algoritmos em tempo real, sem instalação de software.

Referências

Omar Al-Salami, Ibrahim Almarashdeh, Mutasem Alsmadi, Ghaith Jaradat, Ahmed Althunibat, Abdullah Alzaqebah, Ahmed Aljaafreh, Rami Mustafa A. Alkhawaldeh, Thamer Shudayfat, and Zakaria Al-Qadi. A systematic review and analysis of intelligence-based pathfinding algorithms in the field of video games. *Applied Sciences*, 12(11):5499, 2022. doi: 10.3390/app12115499. URL <https://www.mdpi.com/2076-3417/12/11/5499>.

- M. Barbehön and P. Chen. An optimal anytime search algorithm. *Expert Systems with Applications*, 9(3):397–405, 1995. doi: 10.1016/0957-4174(95)00023-9.
- Adi Botea, Martin Müller, and Jonathan Schaeffer. Near optimal hierarchical path-finding. *Journal of Game Development*, 1(1):7–28, 2004.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edition, 2009.
- Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.
- Daniel Harabor and Alban Grastien. Online graph pruning for pathfinding on grid maps. In *Proceedings of the 25th AAAI Conference on Artificial Intelligence*, volume 25, pages 1114–1119. AAAI Press, 2011.
- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.
- Jesper Juul. *Half-Real: Video Games between Real Rules and Fictional Worlds*. MIT Press, Cambridge, MA, 2005.
- Sven Koenig and Maxim Likhachev. D*lite. In *Proceedings of the 18th National Conference on Artificial Intelligence (AAAI)*, pages 476–483. AAAI Press, 2002.
- Richard E. Korf. Real-time heuristic search. *Artificial Intelligence*, 42(2–3):189–211, 1990. doi: 10.1016/0004-3702(90)90054-4.
- Ian Millington and John Funge. *Artificial Intelligence for Games*. Morgan Kaufmann, Burlington, MA, 2nd edition, 2009.
- J. Y. Moon, M. P. Silva, and L. Chaimowicz. Diversifying dynamic difficulty adjustment agent by behavioral modes. *Expert Systems with Applications*, 204:117597, 2022. doi: 10.1016/j.eswa.2022.117597.
- Ira Pohl. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3–4):193–204, 1970. doi: 10.1016/0004-3702(70)90007-X.
- Steve Rabin. A* optimizations. In Steve Rabin, editor, *Game AI Pro 2: Collected Wisdom of Game AI Professionals*, pages 113–130. CRC Press, Boca Raton, FL, 2015.
- R. Rahim, D. Abdullah, S. Nurarif, and M. Ramadhan. Breadth first search approach for shortest path solution in cartesian area. *Journal of Physics: Conference Series*, 1019(1):012038, 2018. doi: 10.1088/1742-6596/1019/1/012038.
- Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, Upper Saddle River, NJ, 3rd edition, 2010.
- Noura Salem, Haneya Abdullah, Aisha Alansari, and Muneera Aljumaiah. Exploring the maze: A comparative study of path finding algorithms for pac-man game. *Effat University Student Research Journal*, 2024. URL <https://repository.effatuniversity.edu.sa/bitstreams/9c0a1948-9926-4c30-8fda-d5c167d68eb3/download>.
- Katie Salen and Eric Zimmerman. *Rules of Play: Game Design Fundamentals*. MIT Press, Cambridge, MA, 2004.
- Robert E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972. ISSN 0097-5397. doi: 10.1137/0201010.
- World Wide Web Consortium. High resolution time level 3. W3C Working Draft, 2023. URL <https://www.w3.org/TR/hr-time-3/>. Especifica a API `performance.now()` para medição de tempo de sub-milissegundo em navegadores web.

Apêndice A — Implementações dos Algoritmos

As listagens a seguir apresentam o código-fonte TypeScript das estruturas de dados e de todos os algoritmos implementados neste trabalho. A lógica de cada algoritmo é descrita em prosa/pseudocódigo no corpo do texto (Seções 3.3 a 3.7); as listagens aqui servem de referência para verificação da implementação.

Listing 1. A.1 — Interface Cell

```
export interface Cell {  
  readonly x: number;  
  readonly y: number;  
  walkable: boolean;  
  hasItem: boolean;  
  readonly worldX: number;  
  readonly worldY: number;  
  f: number;  
  g: number;  
  h: number;  
  parent: Cell | null;  
}
```

Listing 2. A.2 — Classe Grid (trecho: construtor e createCells)

```
export class Grid {  
  public rows: number;  
  public cols: number;  
  private cellSize: number;  
  public cells: Cell[][];  
  
  constructor(rows: number, cols: number, cellSize: number) {  
    this.rows = rows;  
    this.cols = cols;  
    this.cellSize = cellSize;  
    this.cells = this.createCells();  
    this.generateObstacles();  
  }  
  
  private createCells(): Cell[][] {  
    const cells: Cell[][] = [];  
    for (let y = 0; y < this.rows; y++) {  
      cells[y] = [];  
      for (let x = 0; x < this.cols; x++) {  
        cells[y][x] = {  
          x, y,  
          walkable: true,  
          worldX: x * this.cellSize,  
          worldY: y * this.cellSize,  
          f: 0, g: 0, h: 0,  
          parent: null,  
        };  
      }  
    }  
  }  
}
```



```

    }
    return cells;
}
// demais metodos: getNeighbors, findPath, generateObstacles
}

```

Listing 3. A.3 — Método dijkstra

```

private dijkstra(start: Cell, goal: Cell): Cell[] | null {
    const open    = new BinaryHeap();
    const closed  = new Set<Cell>();
    const seen    = new Set<Cell>();
    let seq       = 0;

    start.g = 0; start.f = 0;
    open.push({ cell: start, key: start.g, seq: seq++ });
    seen.add(start);

    while (open.size > 0) {
        const current = open.pop()!.cell;
        if (closed.has(current)) continue;           // entrada obsoleta
                                                    (lazy deletion)
        if (current === goal) return this.buildPath(current);

        closed.add(current);
        this.nodesVisited++;

        for (const nb of this.grid.getNeighbors(current)) {
            if (closed.has(nb)) continue;
            const g = current.g + 1;
            if (!seen.has(nb) || g < nb.g) {
                nb.parent = current; nb.g = g; nb.f = g;
                seen.add(nb);
                open.push({ cell: nb, key: nb.g, seq: seq++ });
            }
        }
    }
    return null;
}

```

Listing 4. A.4 — Método bfs

```

private bfs(start: Cell, goal: Cell): Cell[] | null {
    // Fila FIFO com indice de head: evita o custo O(n) do
    // Array.prototype.shift() em JavaScript e preserva a
    // complexidade canonica O(V + E) do BFS.
    const queue: Cell[] = [start];
    let head = 0;
    const visited: Set<Cell> = new Set([start]);
    while (head < queue.length) {
        const current = queue[head++];
    }
}

```

```

    if (current === goal) return this.buildPath(current);
    this.nodesVisited++;
    for (const nb of this.grid.getNeighbors(current)) {
        if (!visited.has(nb)) {
            visited.add(nb);
            nb.parent = current;
            queue.push(nb);
        }
    }
}
return null;
}

```

Listing 5. A.5 — Método dfs (variante *Greedy DFS*)

```

private dfs(start: Cell, goal: Cell): Cell[] | null {
    const MAX_DEPTH = this.grid.rows * this.grid.cols;
    const stack: { cell: Cell; depth: number }[] =
        [{ cell: start, depth: 0 }];
    const visited: Set<Cell> = new Set([start]);
    const cameFrom: Map<Cell, Cell> = new Map();
    while (stack.length > 0) {
        const { cell: current, depth } = stack.pop()!;
        if (current === goal) {
            const path: Cell[] = [];
            let tmp: Cell | null = current;
            while (tmp) { path.push(tmp); tmp = cameFrom.get(tmp) ??
                null; }
            return path.reverse();
        }
        if (depth < MAX_DEPTH) {
            this.nodesVisited++;
            // Ordena vizinhos do mais distante ao mais proximo (LIFO)
            // para que o mais proximo do objetivo seja expandido
            // primeiro.
            const neighbors = this.grid.getNeighbors(current)
                .sort((a, b) =>
                    this.heuristic(b, goal) - this.heuristic(a, goal));
            for (const nb of neighbors) {
                if (!visited.has(nb)) {
                    visited.add(nb);
                    cameFrom.set(nb, current);
                    stack.push({ cell: nb, depth: depth + 1 });
                }
            }
        }
    }
    return null;
}

```

Listing 6. A.6 — Método aStar

```
private aStar(start: Cell, goal: Cell): Cell[] | null {
  const open = new BinaryHeap();
  const closed = new Set<Cell>();
  const seen = new Set<Cell>();
  let seq = 0;

  start.g = 0;
  start.h = this.heuristic(start, goal);
  start.f = start.g + start.h;
  open.push({ cell: start, key: start.f, seq: seq++ });
  seen.add(start);

  while (open.size > 0) {
    const current = open.pop()!.cell;
    if (closed.has(current)) continue;
    if (current === goal) return this.buildPath(current);

    closed.add(current);
    this.nodesVisited++;

    for (const nb of this.grid.getNeighbors(current)) {
      if (closed.has(nb)) continue;
      const g = current.g + 1;
      if (!seen.has(nb) || g < nb.g) {
        nb.parent = current;
        nb.g = g;
        nb.h = this.heuristic(nb, goal);
        nb.f = nb.g + nb.h;
        seen.add(nb);
        open.push({ cell: nb, key: nb.f, seq: seq++ });
      }
    }
  }
  return null;
}
```

Listing 7. A.7 — Método jump do JPS

```
private jpsJump(
  x: number, y: number,
  dx: number, dy: number,
  goal: Cell
): Cell | null {
  let cx = x + dx, cy = y + dy;
  while (true) {
    if (!this.grid.isValid(cx, cy) ||
        !this.grid.cells[cy][cx].walkable) return null;
    const cell = this.grid.cells[cy][cx];
    if (cell === goal) return cell;
  }
}
```

```

    if (this.jpsHasForced(cx, cy, dx, dy)) return cell;
    // Sondagem perpendicular linear (sem recursao) --
    // necessaria
    // em grades 4-direcionais para detectar pontos de salto em
    // corredores sem obstaculos adjacentes ao no atual.
    if (dx !== 0) {
        if (this.jpsProbe(cx, cy, 0, 1, goal)) return cell;
        if (this.jpsProbe(cx, cy, 0, -1, goal)) return cell;
    } else {
        if (this.jpsProbe(cx, cy, 1, 0, goal)) return cell;
        if (this.jpsProbe(cx, cy, -1, 0, goal)) return cell;
    }
    cx += dx; cy += dy;
}
}

```

Listing 8. A.8 — Detecção de vizinhos forçados

```

private jpsHasForced(
  x: number, y: number,
  dx: number, dy: number
): boolean {
  if (dx !== 0) {
    if (this.walkable(x, y+1) &&
        !this.walkable(x-dx, y+1)) return true;
    if (this.walkable(x, y-1) &&
        !this.walkable(x-dx, y-1)) return true;
  } else {
    if (this.walkable(x+1, y) &&
        !this.walkable(x+1, y-dy)) return true;
    if (this.walkable(x-1, y) &&
        !this.walkable(x-1, y-dy)) return true;
  }
  return false;
}

```

Listing 9. A.9 — Fila de prioridade (heap binário mínimo)

```

interface HeapEntry { cell: Cell; key: number; seq: number; }

export class BinaryHeap {
  private readonly items: HeapEntry[] = [];
  get size(): number { return this.items.length; }

  push(entry: HeapEntry): void {
    const a = this.items; a.push(entry);
    let i = a.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (this.less(a[i], a[p])) { const t = a[i]; a[i] = a[p];
        a[p] = t; i = p; }
    }
  }
}

```

```

        else break;
    }
}

pop(): HeapEntry | undefined {
    const a = this.items, n = a.length;
    if (n === 0) return undefined;
    const top = a[0], last = a.pop()!;
    if (n > 1) {
        a[0] = last; const len = a.length; let i = 0;
        while (true) {
            const l = 2 * i + 1, r = l + 1; let m = i;
            if (l < len && this.less(a[l], a[m])) m = l;
            if (r < len && this.less(a[r], a[m])) m = r;
            if (m === i) break;
            const t = a[i]; a[i] = a[m]; a[m] = t; i = m;
        }
    }
    return top;
}

// menor key primeiro; empate pela menor sequencia de insercao
// (desempate estavel)
private less(x: HeapEntry, y: HeapEntry): boolean {
    return x.key < y.key || (x.key === y.key && x.seq < y.seq);
}
}

```